



Université Joseph Ki-ZERBO



Unité de formation et de recherche en
science exacte et Appliquées (UFR/SEA)

RAPPORT DE PROJET

Projet d'API Services Web

Tronc commun Master 1 Informatique

Membre du groupe :

KOALA Issouf

KONATE Abdoul Kayoum

NATCHIELE DABIRE Sag-Woulou

SORGHO Abdoul Aziz

Enseignant :

Dr. SOMDA

Année académique : 2024-2025

INTRODUCTION

Le projet présenté ici a pour objectif de développer un système de gestion de tâches basé sur une API REST. Ce système permettra de réaliser les opérations de base telles que la création, la lecture, la mise à jour et la suppression (CRUD) des tâches. Conçu en utilisant le langage de programmation PHP et un serveur Apache, ce projet vise à offrir une solution robuste et modulable pour la gestion de tâches dans des contextes variés, allant des projets individuels aux environnements d'entreprise.

Pour aller au-delà de la simple gestion des tâches, nous proposons une extension de ce projet qui consiste à développer une autre API REST complètement indépendante pour la gestion des employés. Cette nouvelle API permettra non seulement de créer et de gérer des données relatives aux employés, mais également de faciliter la communication entre les deux systèmes. Par exemple, chaque tâche pourra être associée à un employé spécifique pour un suivi plus efficace.

Le choix de PHP comme langage principal repose sur sa popularité, sa facilité d'utilisation et sa compatibilité étroite avec Apache. En utilisant le Framework Laravel, nous allons accélérer le développement, garantir une meilleure sécurité.

En développant ces systèmes interconnectés, ce projet vise à démontrer comment des API REST indépendantes peuvent collaborer pour répondre à des besoins plus complexes. Ce rapport expliquera les détails techniques, les choix d'architecture et les résultats obtenus à chaque étape du projet.

Configuration du Projet API avec Laravel

Laravel

Laravel est un framework PHP open-source conçu pour simplifier le développement d'applications web. Basé sur l'architecture MVC (Modèle-Vue-Contrôleur), il offre des fonctionnalités avancées telles que :

- ORM Eloquent : Gestion simplifiée des bases de données.
- Moteur de templates Blade : Création dynamique des vues.
- Gestion des migrations : Suivi des modifications de la structure de la base de données.

Laravel est apprécié pour sa simplicité, sa documentation exhaustive, et son écosystème riche.

Présentation des Projets des API

Nous avons développé deux projets indépendants :

1. API de gestion des tâches

- **Port** : 8080
- **Base de données** : gestionTaches
- **Création de la table** taches

```
CREATE TABLE taches (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  titre VARCHAR(255) NOT NULL,  
  description TEXT NOT NULL,  
  dateEcheance DATE NOT NULL,  
  statut VARCHAR(50) NOT NULL,  
  id_employe INT  
);
```

Ensuite dans le fichier **.env** pour l'API Gestion des Tâches configurer la connexion à la base de données :

```
DB_CONNECTION=mysql  
DB_HOST=127.0.0.1  
DB_PORT=3306  
DB_DATABASE=gestionTaches  
DB_USERNAME=root  
DB_PASSWORD=
```

2. API de gestion des employés

- **Port** : 8081
- **Base de données** : gestionEmployes
- **Création de la tables** employees

```
CREATE TABLE employees (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nom VARCHAR(255) NOT NULL,  
    poste VARCHAR(255) NOT NULL,  
    disponibilite VARCHAR(50) NOT NULL,  
    date_creation TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    created_at TIMESTAMP NULL DEFAULT NULL,  
    updated_at TIMESTAMP NULL DEFAULT NULL  
);
```

Ensuite dans le fichier **.env** pour l'API Gestion des Employés configurer la connexion à la base de données :

```
DB_CONNECTION=mysql  
DB_HOST=127.0.0.1  
DB_PORT=3306  
DB_DATABASE=gestionEmployes  
DB_USERNAME=root  
DB_PASSWORD=
```

Installation et Configuration de l'Environnement

Prérequis

- PHP >= 8.0
- Composer
- Serveur Web (Apache ou Nginx)
- MySQL ou MariaDB
- Postman (pour tester les APIs)

Création des projets

```
composer create-project --prefer-dist laravel/laravel api-gestion-taches  
composer create-project --prefer-dist laravel/laravel api-gestion-employes
```

I. Développement de l'API

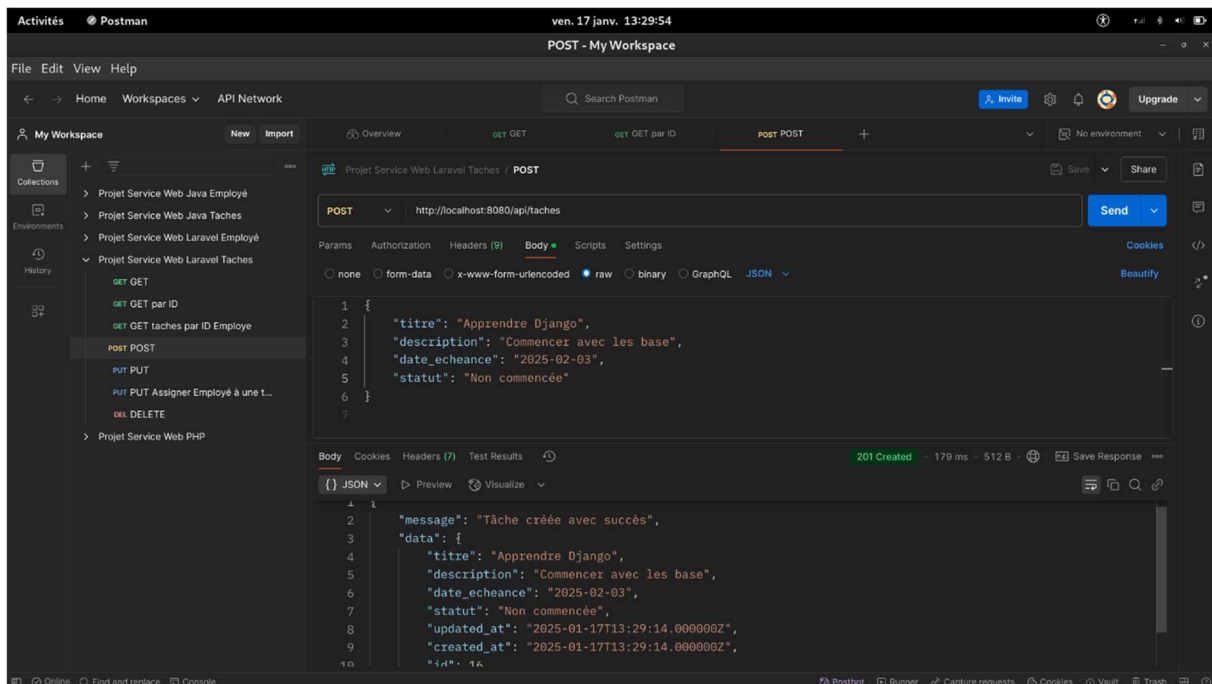
1. POST : Création d'une tâche

Cette opération consiste à ajouter une nouvelle tâche dans la base de données. Elle s'appuie sur la méthode HTTP POST et reçoit des données structurées au format JSON. Ces données contiennent les détails essentiels de la tâche, notamment son titre, sa description, sa date d'échéance et son statut.

Code source

```
13 // Créer une tâche
14 public function store(Request $request) {
15     $validated = $request->validate([
16         'titre' => 'required|string|max:50',
17         'description' => 'required|string|max:50',
18         'date_echeance' => 'required|date',
19         'statut' => 'required|string',
20     ]);
21
22     $tache = Tache::create($validated);
23     return response()->json(['message' => 'Tâche créée avec succès', 'data' => $tache], 201);
24 }
```

Test sur Postman



2. GET : Récupérer une tâche grâce à son ID

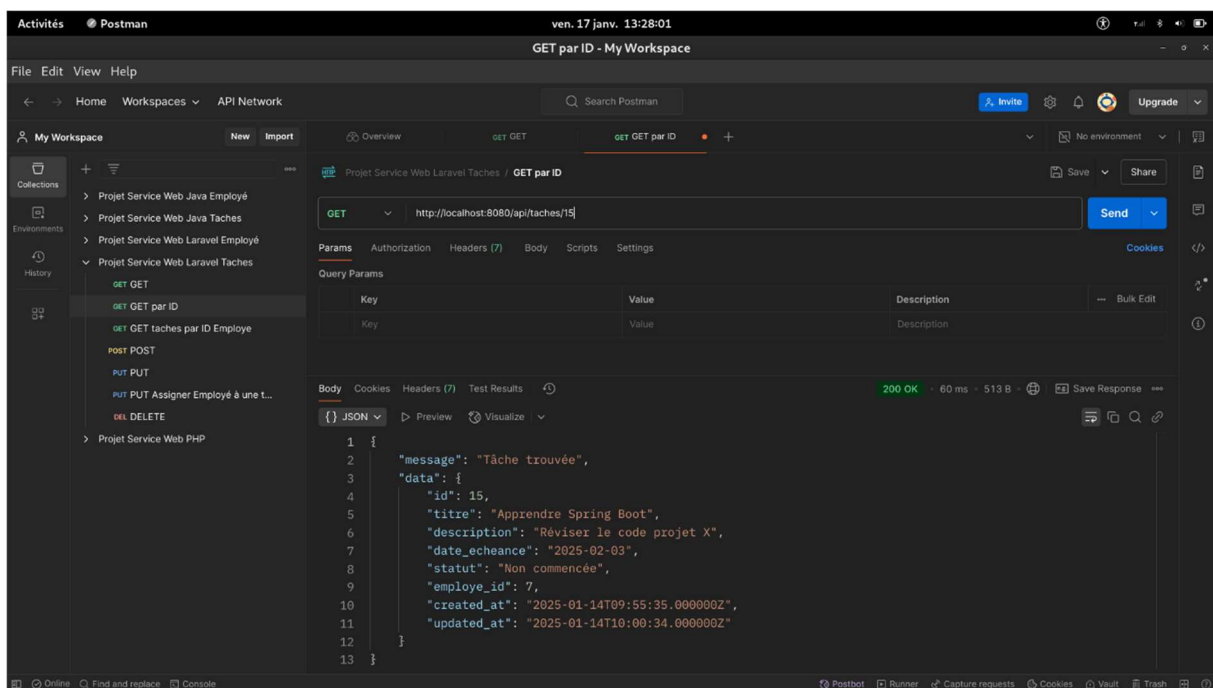
Cette opération consiste à récupérer les tâches enregistrées dans la base de données. Le script vérifie si un paramètre id est fourni dans l'URL. Si oui, il retourne les informations de la tâche

correspondante. Sinon, il renvoie un message d'erreur 'la tâche n'a pas été trouvée '. Les données sont retournées au format JSON.

Code source

```
// Obtenir une tâche par ID
Codeium: Refactor | Explain | X
public function show($id) {
    $tache = Tache::find($id);
    if (!$tache) {
        return response()->json(['message' => 'Tâche non trouvée'], 404);
    }
    return response()->json(['message' => 'Tâche trouvée', 'data' => $tache], 200);
}
```

Test sur Postman



3. PUT : Mise à jour d'une tâche

Le script reçoit des données JSON spécifiant la tâche à modifier ainsi que ses nouveaux détails, puis met à jour les informations correspondantes dans la base de données. Une requête préparée est utilisée afin de garantir la sécurité des données.

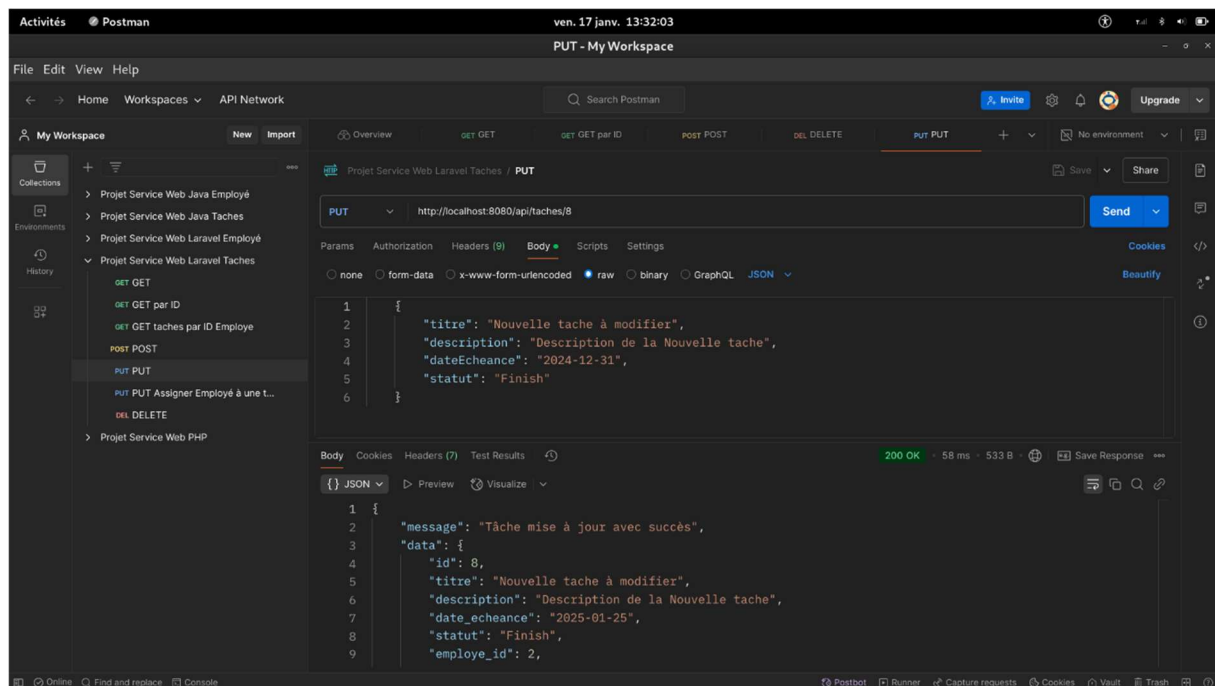
Code source

```
// Mettre à jour une tâche
Codeium: Refactor | Explain | X
public function update(Request $request, $id) {
    $tache = Tache::find($id);
    if (!$tache) {
        return response()->json(['message' => 'Tâche non trouvée'], 404);
    }

    $validated = $request->validate([
        'titre' => 'string|max:50',
        'description' => 'string|max:50',
        'date_echeance' => 'date',
        'statut' => 'string',
    ]);

    $tache->update($validated);
    return response()->json(['message' => 'Tâche mise à jour avec succès', 'data' => $tache], 200);
}
```

Test sur Postman



4. DELETE : Suppression d'une tâche

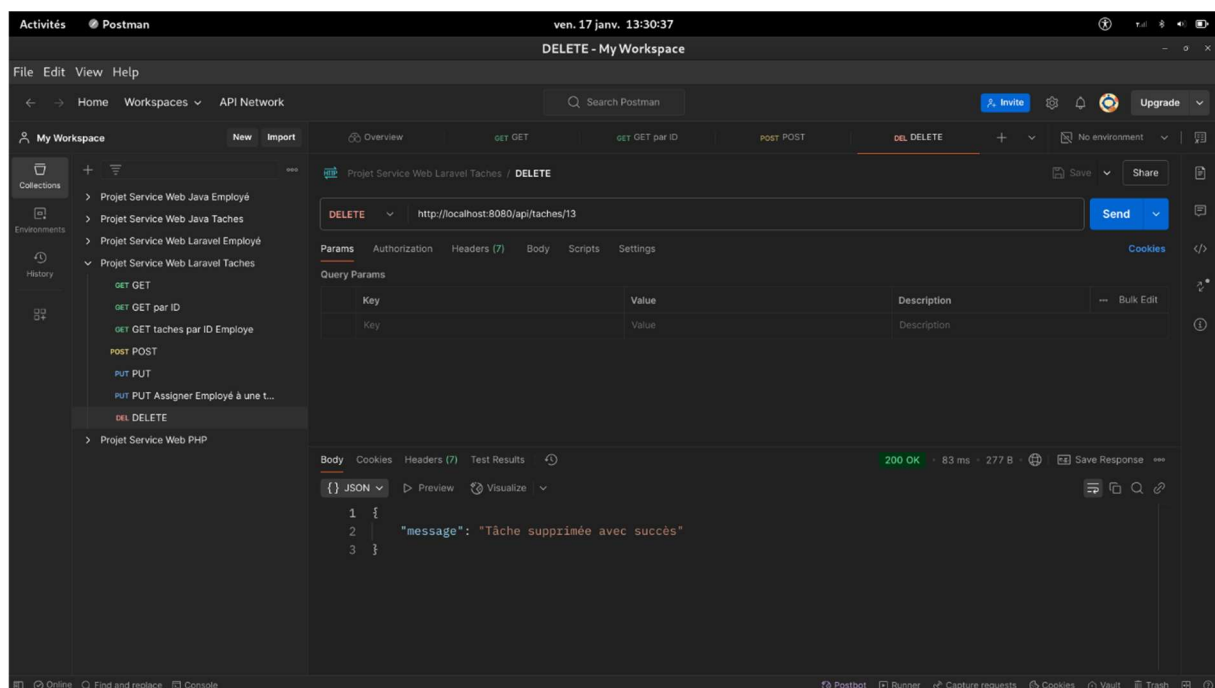
Cette opération utilise la méthode DELETE pour supprimer une tâche spécifique de la base de données. L'identification de la tâche à supprimer se fait à l'aide de son ID. En cas de succès, la tâche est définitivement retirée de la base de données.

Code source

```
// Supprimer une tâche
Codeium: Refactor | Explain | X
public function destroy($id) {
    $tache = Tache::find($id);

    if ($tache) {
        $tache->delete();
        return response()->json(['message' => 'Tâche supprimée avec succès']);
    } else {
        return response()->json(['message' => 'Tâche non trouvée'], 404);
    }
}
```

Test sur Postman



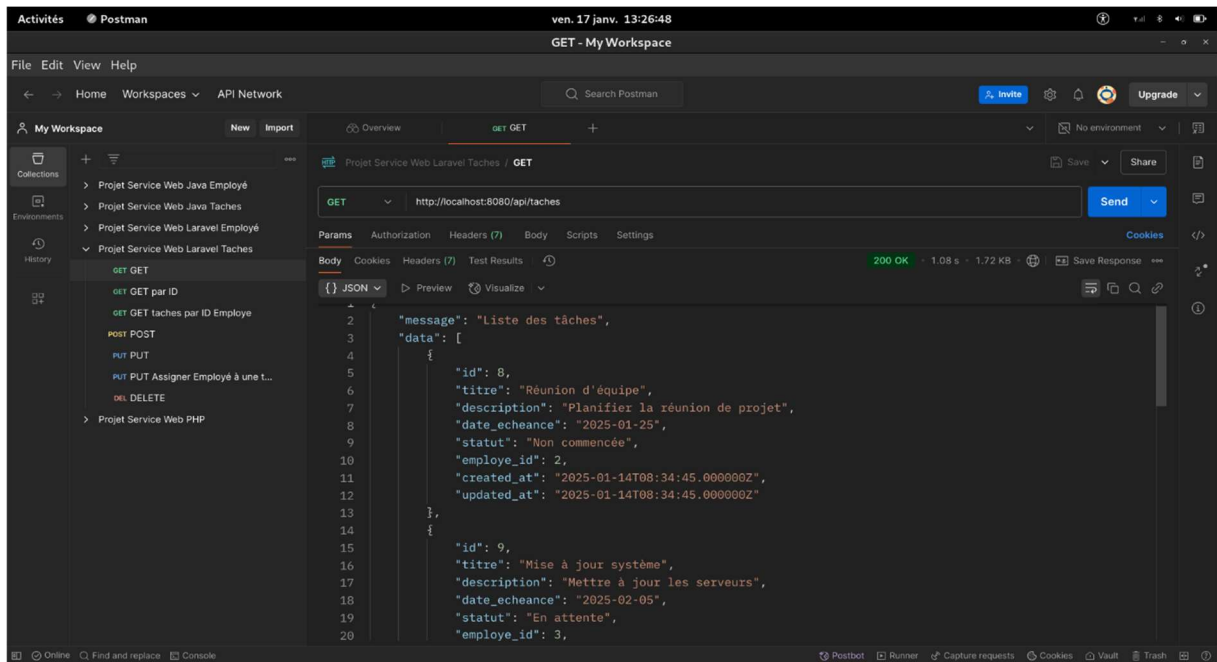
5. GET : Récupérer toutes les taches

Cette opération consiste à récupérer toutes les tâches enregistrées dans la base de données. Les données sont retournées au format JSON.

Code source



Test Sur Postman



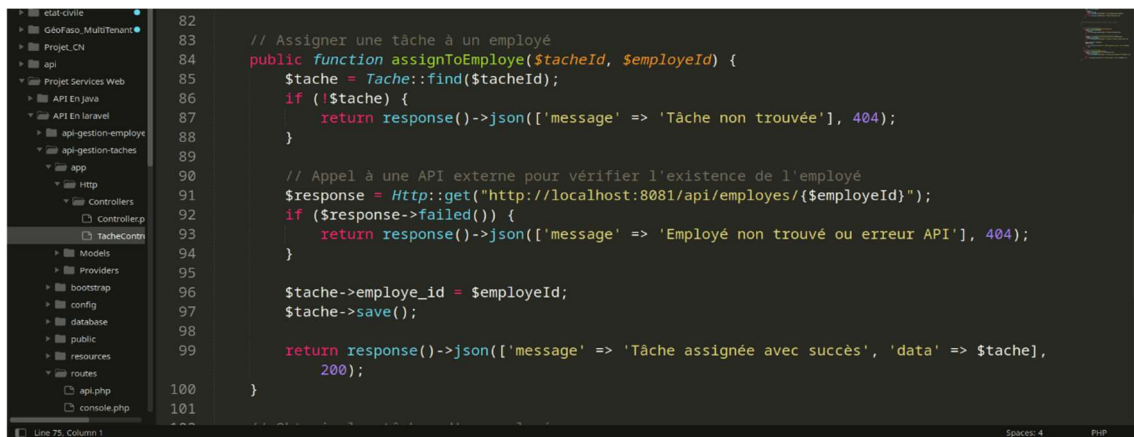
NB : Nous avons exactement les avons également 5 API Fonctionnelle pour la gestion des Employés

II. Communication entre les deux APIs

1. Assigner une tâche à un Employé en étant dans l'API `tache`

Cette opération permet d'assigner une tâche à un employé en utilisant la méthode HTTP PUT. Les données de la requête JSON incluent l'ID de la tâche et l'identifiant de l'employé. En cas de succès, un message de confirmation est retourné au format JSON, tandis qu'en cas d'erreur, un message explicatif est fourni.

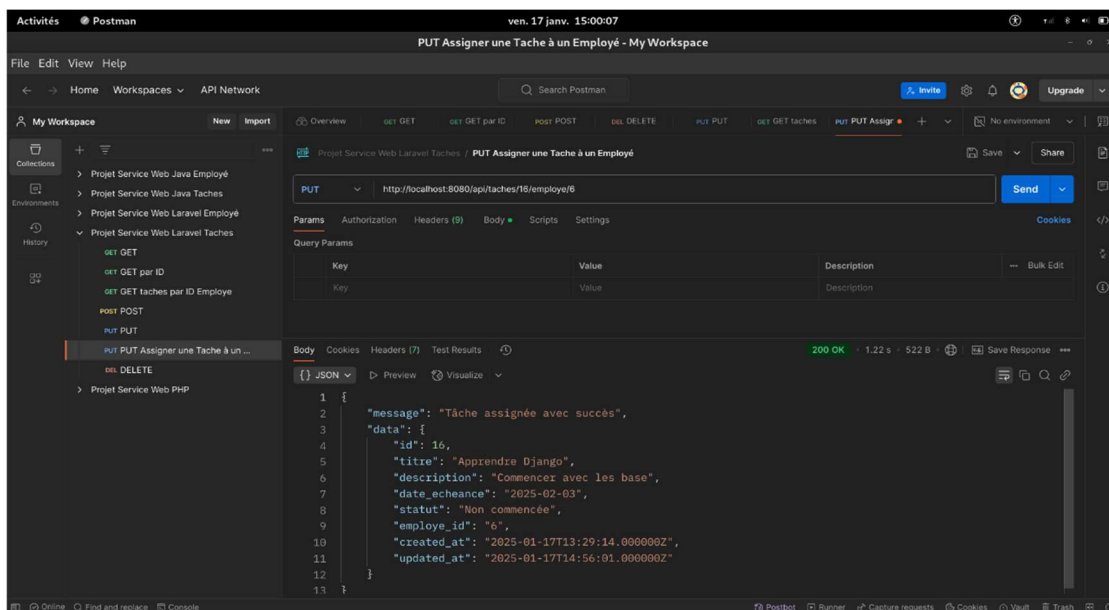
Code source



```
82
83
84 // Assigner une tâche à un employé
85 public function assignToEmployee($tacheId, $employeeId) {
86     $tache = Tache::find($tacheId);
87     if (!$tache) {
88         return response()->json(['message' => 'Tâche non trouvée'], 404);
89     }
90
91     // Appel à une API externe pour vérifier l'existence de l'employé
92     $response = Http::get("http://localhost:8081/api/employees/{$employeeId}");
93     if ($response->failed()) {
94         return response()->json(['message' => 'Employé non trouvé ou erreur API'], 404);
95     }
96
97     $tache->employee_id = $employeeId;
98     $tache->save();
99
100    return response()->json(['message' => 'Tâche assignée avec succès', 'data' => $tache],
101                             200);
102 }
```

Ici, l'api de gestion des taches consomme l'api de récupération d'un employé (par son identifiant) afin de pouvoir vérifier l'existence de l'employé et lui assigner sa la tâche.

Test sur Postman



Remarque : Dans l'exemple ci-dessous, nous assignons la tache dont l'id est 16, à l'employé dont l'id est 6.

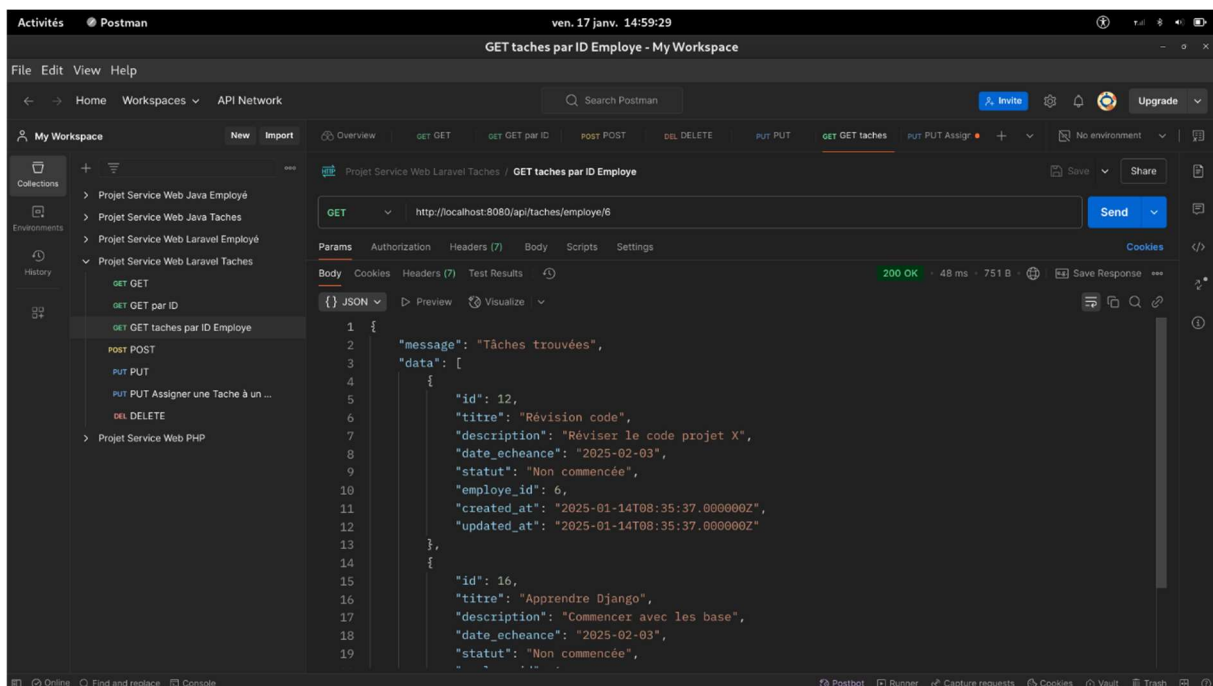
2. Récupérer toutes les tâches d'un employé (en étant dans l'api des tâches)

Cette opération permet de récupérer toutes les tâches associées à un employé spécifique en utilisant la méthode HTTP GET. L'identifiant de l'employé est transmis en paramètre dans l'URL ou la requête. Une fois l'identifiant validé, une requête est exécutée pour extraire toutes les tâches liées à cet employé dans la base de données. Les résultats sont retournés au format JSON, offrant une liste détaillée des tâches, y compris leur titre, description, statut et date d'échéance. En cas d'erreur ou si aucune tâche n'est trouvée, un message approprié est également retourné.

Code source

```
101 // Obtenir les tâches d'un employé
102 public function getByEmployeId($employeId) {
103     $taches = Tache::where('employee_id', $employeId)->get();
104     if ($taches->isEmpty()) {
105         return response()->json(['message' => 'Aucune tâche trouvée pour cet employé'], 404);
106     }
107     return response()->json(['message' => 'Tâches trouvées', 'data' => $taches], 200);
108 }
109 }
110 }
111 }
```

Test Postman



3. Récupérer toutes les tâches d'un employé (en étant dans l'api des employés)

Code source

```
88
89 public function getTachesByEmploye($id) {
90     try {
91         $response = Http::get("http://localhost:8080/api/taches/employee/{$id}");
92
93         if ($response->successful()) {
94             return response()->json(['message' => 'Liste des tâches', 'taches' => $response->
95                 json()]);
96         } else {
97             return response()->json(['message' => 'Tâches non trouvées pour cet employé'], 404
98             );
99         }
100     } catch (\Exception $e) {
101         return response()->json(['message' => 'Erreur lors de la récupération des tâches', '
102             error' => $e->getMessage()], 500);
103     }
104 }
```

Test sur Postman

The screenshot shows the Postman interface with a workspace named "GET les taches d'un employé - My Workspace". The active request is a GET request to the URL "http://localhost:8081/api/employees/3/taches". The response is a 200 OK status with a response time of 207 ms and a body size of 563 B. The response body is displayed in JSON format:

```
{
  "message": "Liste des tâches",
  "taches": {
    "message": "Tâches trouvées",
    "data": [
      {
        "id": 9,
        "titre": "Mise à jour système",
        "description": "Mettre à jour les serveurs",
        "date_echeance": "2025-02-05",
        "statut": "En attente",
        "employee_id": 3,
        "created_at": "2025-01-14T08:34:54.000000Z",
        "updated_at": "2025-01-14T08:34:54.000000Z"
      }
    ]
  }
}
```

CONCLUSION

En conclusion, ce projet a permis de concevoir et de développer un système de gestion de tâches basé sur une API REST, accompagné d'une extension visant à gérer les données des employés. Nous avons exploré et mis en pratique les opérations CRUD fondamentales pour les tâches, tout en étendant le champ d'application à une gestion plus complète, grâce à l'interaction entre deux API REST indépendantes.

Tout au long de ce projet, nous avons appris à maîtriser des concepts clés, tels que la conception d'API REST, l'utilisation du framework Laravel pour le développement rapide et sécurisé. Ce travail nous a également permis de mieux comprendre les défis liés à la communication entre services, l'optimisation des performances et la gestion de la sécurité des données sensibles.

Les perspectives pour ce projet sont nombreuses. Parmi elles, nous pouvons envisager l'ajout de fonctionnalités avancées, telles que la gestion des droits d'accès pour les utilisateurs, la mise en place de tableaux de bord analytiques pour un suivi en temps réel, ou encore l'intégration avec d'autres systèmes externes, comme des outils de gestion de projets ou des plateformes de collaboration. De plus, le passage à une architecture cloud native pourrait également être exploré afin d'améliorer la scalabilité et la résilience des systèmes.